ĐẠI HỌC BÁCH KHOA HÀ NỘI

BÁO CÁO ĐỒ ÁN MÔN HỌC

XÂY DỰNG HỆ THỐNG THU THẬP DỮ LIỆU IOT CHO GIÁM SÁT MÔI TRƯỜNG VÀ CẢNH BÁO AN TOÀN

Học phần:	Project I
Mã lớp học:	755566
Giảng viên hướng dẫn:	Nguyễn Minh Quân
Sinh viên thực hiện:	Hoàng Minh Tuấn

HÀ NỘI, THÁNG 1/2026

MỤC LỤC

TÓM TẮT 3
CHƯƠNG 1 TỔNG QUAN VỀ ĐỀ TÀI 4
1.1 Đặt vấn đề và tính cấp thiết 4
1.2 Mục tiêu nghiên cứu 4
1.3 Phạm vi và đối tượng nghiên cứu 5
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ 6
2.1 Tổng quan về Internet of Things (IoT) 6
2.2 Giao thức MQTT (Message Queuing Telemetry Transport) 6
2.2.1 Khái niệm 6
2.2.2 Mô hình Publish/Subscribe 6
2.3 Vi điều khiển ESP32 6
2.4 Cảm biến DHT22 7
2.5 Cơ sở dữ liệu InfluxDB 7
2.6 Nền tảng Docker và Ảo hóa ứng dụng 7
2.6.1 Khái niệm 7
2.6.2 Ưu điểm so với máy ảo truyền thống 7
2.6.3 Áp dụng trong đồ án 8
2.7 Nền tảng lập trình Android và Dịch vụ nền (Background Service) 8
2.7.1 Ngôn ngữ Kotlin và Kiến trúc ứng dụng 8
2.7.2 Cơ chế Foreground Service 8
2.7.3 Xử lý bất đồng bộ (Asynchronous Processing) 9
CHƯƠNG 3 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 10
3.1 Kiến trúc hệ thống 10
3.2 Thiết kế chi tiết phần cứng 11
3.3 Kiến trúc hệ thống Backend và Luồng dữ liệu (Data flow) 12
3.3.1 Sơ đồ luồng dữ liệu (Data Flow Diagram) 12
3.3.2 Thiết kế chi tiết các Service 13
3.4 Kiến trúc phần mềm Android 13
3.4.1 Mô hình Single Activity - Multiple Fragments 13

3.4.2 Thiết kế luồng xử lý nền và Cảnh báo thời gian thực 13
3.4.3 Tích hợp Trực quan hóa Dữ liệu (WebView Integration) 14
3.4.4 Cơ chế Đồng bộ Trạng thái (State Synchronization) 14
CHƯƠNG 4 XÂY DỰNG THUẬT TOÁN VÀ CÀI ĐẶT 15
4.1 Giải thuật điều khiển trên Node cảm biến (ESP32) 15
4.1.1 Lưu đồ thuật toán tổng quát 15
4.1.2 Cơ chế ghi dữ liệu ngoại tuyến (Data Persistence) 15
4.1.3 Thuật toán đồng bộ lại dữ liệu (Data Synchronization) 16
4.2 Xử lý luồng dữ liệu tại Backend (Node-RED) 16
4.2.1 Quy trình xử lý dữ liệu thời gian thực 16
4.2.2 Thuật toán hợp nhất dữ liệu (Data Aggregation Algorithm) 16
4.3 Giải thuật và quy trình xử lý ứng dụng Android 17
4.3.1 Giải thuật Giám sát và Cảnh báo nền 17
4.3.2 Logic Kiểm tra An toàn Tổng hợp 18
4.3.3 Cơ chế Đồng bộ hóa Cấu hình 18
4.3.4 Giải thuật Tích hợp Hiển thị Lai 18
CHƯƠNG 5 KẾT QUẢ THỰC HIỆN VÀ ĐÁNH GIÁ 19
5.1 Kết quả phần cứng 19
5.2 Kết quả phần mềm 20
5.2.1 Hệ thống Backend và Xử lý dữ liệu 20
5.2.2 Ứng dụng Android 20
5.3 Đánh giá hiệu năng hệ thống 22
CHƯƠNG 6 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 23
6.1 Kết luận 23
6.2 Hạn chế 23
6.3 Hướng phát triển 24
TÀI LIỆU THAM KHẢO 25

TÓM TẮT

Trong bối cảnh chuyển đổi số giáo dục, việc duy trì điều kiện vi khí hậu tiêu chuẩn tại các phòng học và phòng thí nghiệm là yếu tố quan trọng để đảm bảo sức khỏe con người và tuổi thọ thiết bị. Tuy nhiên, các giải pháp IoT thương mại hiện nay thường phụ thuộc vào nền tảng bên thứ ba (SaaS), hạn chế khả năng tùy biến sâu và tiềm ẩn rủi ro về quyền riêng tư dữ liệu. Xuất phát từ thực tế đó, đồ án tập trung nghiên cứu và phát triển một hệ thống giám sát môi trường khép kín, tự chủ hoàn toàn về công nghệ (Self-hosted) từ phần cứng đến phần mềm.

Về phần cứng, hệ thống sử dụng vi điều khiển **ESP32** tích hợp kết nối WiFi, kết hợp với cảm biến **DHT22** độ chính xác cao và màn hình OLED hiển thị tại chỗ. Thiết bị được thiết kế hoạt động độc lập nhờ module nguồn pin **18650 Battery Shield**, đảm bảo tính di động cao. Giao thức truyền thông **MQTT** (Message Queuing Telemetry Transport) được lựa chọn làm nòng cốt nhờ ưu điểm tiết kiệm băng thông và năng lượng.

Về phần mềm, hệ thống Backend được xây dựng trên kiến trúc Microservices cục bộ gồm **Mosquitto Broker** (trung chuyển tin nhắn), **Node-RED** (xử lý logic), **InfluxDB** (lưu trữ chuỗi thời gian) và **Grafana** (trực quan hóa). Điểm nhấn của đồ án là ứng dụng di động **Android Native (Kotlin)** được phát triển với kiến trúc Service, sử dụng **Foreground Service** để duy trì kết nối ngầm, đảm bảo nhận cảnh báo rung và thông báo đẩy ngay cả khi điện thoại ở trạng thái nghỉ.

Kết quả thực nghiệm cho thấy hệ thống hoạt động ổn định với độ trễ thấp (dưới 500ms trong mạng LAN). Đồ án đã giải quyết thành công các thách thức kỹ thuật quan trọng như: khắc phục hiện tượng sụt áp (Brownout) khi khởi động ESP32, xây dựng thuật toán đồng bộ dữ liệu lệch thời gian (Data Merging) để xuất báo cáo Excel chuẩn hóa, và tích hợp biểu đồ lịch sử trực quan ngay trên ứng dụng di động.

Từ khóa: *IoT, ESP32, MQTT, Android Kotlin, Foreground Service, Node-RED, InfluxDB, Giám sát môi trường.*

CHƯƠNG 1 TỔNG QUAN VỀ ĐỀ TÀI

1.1 Đặt vấn đề và tính cấp thiết

Môi trường phòng học, đặc biệt là tại Việt Nam với khí hậu nhiệt đới gió mùa nóng ẩm, thường xuyên đối mặt với những biến động lớn về nhiệt độ và độ ẩm.

- **Về nhiệt độ:** Nhiệt độ quá cao (>35°C) gây mệt mỏi, mất tập trung cho người học. Ngược lại, việc quên tắt điều hòa sau giờ học gây lãng phí năng lượng lớn.
- **Về độ ẩm:** Độ ẩm cao (>80%) là kẻ thù của các thiết bị điện tử trong phòng thực hành máy tính, gây oxy hóa linh kiện, chập cháy mainboard và tạo điều kiện cho nấm mốc phát triển.

Các giải pháp IoT hiện có trên thị trường như sử dụng Blynk, ThingSpeak hay Google Firebase thường bộc lộ các nhược điểm sau:

- **Phụ thuộc bên thứ ba:** Dữ liệu phải đi vòng qua Server nước ngoài, gây độ trễ (latency) cao và rủi ro gián đoạn dịch vụ.
- **Hạn chế tùy chỉnh:** Không cho phép can thiệp sâu vào luồng xử lý dữ liệu (Data Pipeline) để thực hiện các thuật toán phức tạp (như gộp dữ liệu, lọc nhiễu chuyên sâu).
- **Chi phí:** Các gói miễn phí thường giới hạn số lượng thiết bị và tần suất gửi tin.

Do đó, việc nghiên cứu xây dựng một hệ thống giám sát tự chủ (Self-hosted) chạy trên hạ tầng mạng nội bộ (Local/Intranet) hoặc VPS riêng là vô cùng cấp thiết.

1.2 Mục tiêu nghiên cứu

Đề tài hướng tới việc giải quyết các bài toán kỹ thuật cốt lõi sau:

- **Xây dựng nút cảm biến (Sensor Node):** Thiết kế thiết bị nhỏ gọn, hoạt động độc lập bằng nguồn pin, có khả năng đo đạc chính xác và hiển thị tại chỗ.

- **Làm chủ giao thức truyền thông:** Ứng dụng giao thức MQTT (Message Queuing Telemetry Transport) để tối ưu băng thông và năng lượng cho thiết bị IoT.

- **Phát triển Backend hiệu năng cao:** Sử dụng Node-RED để xử lý luồng dữ liệu và InfluxDB để lưu trữ dữ liệu chuỗi thời gian (Time-series) với khả năng truy xuất hàng triệu bản ghi trong tích tắc.

- **Phát triển ứng dụng di động (Mobile App):** Xây dựng App Android Native có khả năng chạy ngầm (Background Service), đảm bảo cảnh báo tức thì ngay cả khi điện thoại ở chế độ ngủ.

1.3 Phạm vi và đối tượng nghiên cứu

- **Đối tượng nghiên cứu:** Các kỹ thuật giao tiếp phần cứng (GPIO, I2C, OneWire), giao thức mạng (TCP/IP, MQTT), cơ sở dữ liệu và lập trình di động.
- **Phạm vi không gian:** Triển khai thử nghiệm tại phòng học/phòng làm việc cá nhân với kết nối WiFi cục bộ.
- **Giới hạn:** Hệ thống tập trung vào giám sát 02 thông số chính là Nhiệt độ và Độ ẩm. Chưa tích hợp điều khiển điều hòa qua hồng ngoại (IR).

CHƯƠNG 2 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

2.1 Tổng quan về Internet of Things (IoT)

IoT là mạng lưới vạn vật kết nối, nơi các thiết bị vật lý, xe cộ, tòa nhà và các trang thiết bị khác được nhúng với các bộ phận điện tử, phần mềm, cảm biến và khả năng kết nối mạng giúp cho các thiết bị này có thể thu thập và trao đổi dữ liệu. Kiến trúc IoT trong đề tài này tuân theo mô hình 4 lớp tiêu chuẩn:

- **Lớp Cảm biến (Sensing Layer):** Thu thập dữ liệu môi trường (DHT22).
- **Lớp Mạng (Network Layer):** Truyền tải dữ liệu qua WiFi/MQTT.
- **Lớp Xử lý dữ liệu (Data Processing Layer):** Node-RED và InfluxDB
- **Lớp Ứng dụng (Application Layer):** App Android và Dashboard Grafana.

2.2 Giao thức MQTT (Message Queuing Telemetry Transport)

2.2.1 Khái niệm

MQTT là giao thức truyền thông điệp theo mô hình Publish/Subscribe (Xuất bản/Đăng ký), được thiết kế cho các kết nối băng thông thấp, độ trễ cao và không tin cậy. Đây là giao thức chuẩn "de facto" trong công nghiệp IoT.

2.2.2 Mô hình Publish/Subscribe

Các thành phần trong mô hình:

- **Publisher**: Gửi Message đến một topic
- **Subscriber:** Đăng ký nhận message từ topic
- **Broker:** Trung gian quản lý và phân phối message
- **Topic:** Kênh định danh cho message

2.3 Vi điều khiển ESP32

ESP32 là dòng vi điều khiển SoC (System on Chip) tích hợp WiFi và Bluetooth 2 chế độ, được phát triển bởi Espressif Systems

Thông số kỹ thuật:

- **CPU:** Xtensa® Dual-Core 32-bit LX6, xung nhịp lên đến 240MHz.
- **Bộ nhớ:** 520 KB SRAM nội bộ, hỗ trợ Flash ngoài lên đến 16MB.
- **Kết nối không dây:** Wi-Fi 802.11 b/g/n (lên đến 150 Mbps) và Bluetooth v4.2 BR/EDR & BLE.
- **Ngoại vi:** 34 GPIO, 3 x UART, 3 x SPI, 2 x I2C, 12 x ADC input, 2 x DAC.
- **Điện áp hoạt động:** 2.2V - 3.6V (Cần mạch ổn áp khi dùng pin 4.2V).

2.4 Cảm biến DHT22

Thông số kỹ thuật:

- **Giao tiếp:** Digital One-wire (1 dây dữ liệu).
- **Dải đo nhiệt độ:** -40°C đến 80°C (Sai số ±0.5°C).
- **Dải đo độ ẩm:** 0% đến 100% RH (Sai số ±2% RH).
- **Tần suất lấy mẫu:** Tối đa 0.5Hz (2 giây/lần). Trong đề tài này, chu kỳ gửi dữ liệu được thiết lập là 5 giây/lần để đảm bảo độ bền.

2.5 Cơ sở dữ liệu InfluxDB

InfluxDB là cơ sở dữ liệu chuỗi thời gian (Time Series Database - TSDB) mã nguồn mở. InfluxDB được tối ưu hóa để ghi nhận dữ liệu có gắn mốc thời gian (Timestamp) với hiệu suất ghi (Write throughput) cực cao, rất phù hợp để lưu trữ dữ liệu cảm biến gửi về liên tục 24/7.

2.6 Nền tảng Docker và Ảo hóa ứng dụng

2.6.1 Khái niệm

Thay vì cài đặt trực tiếp phần mềm lên hệ điều hành (Windows/Linux) vốn tiềm ẩn nhiều rủi ro xung đột thư viện và khó quản lý phiên bản, đồ án sử dụng công nghệ Docker để "đóng gói" các ứng dụng.
Docker hoạt động dựa trên cơ chế ảo hóa mức hệ điều hành (OS-level virtualization). Nó cho phép chạy các ứng dụng trong các môi trường cách ly gọi là "Container". Mỗi Container chứa đầy đủ thư viện và cấu hình cần thiết để ứng dụng chạy được mà không phụ thuộc vào máy chủ vật lý.

2.6.2 Ưu điểm so với máy ảo truyền thống

Tiết kiệm tài nguyên: Không cần giả lập cả một hệ điều hành khách (Guest OS) nặng nề như VMware hay VirtualBox.

Triển khai nhanh: Các Container (như InfluxDB, Grafana) có thể được khởi chạy hoặc tắt đi chỉ trong vài giây.

Tính nhất quán: Đảm bảo môi trường chạy thử nghiệm trên máy cá nhân giống hệt với môi trường chạy thật, đảm bảo tính nhất quán giữa môi trường phát triển và môi trường triển khai

2.6.3 Áp dụng trong đồ án

Trong hệ thống này, nhóm thực hiện sử dụng Docker để triển khai hai thành phần quan trọng của Backend:

- **InfluxDB Container:** Chịu trách nhiệm lưu trữ cơ sở dữ liệu chuỗi thời gian, lắng nghe ở cổng 8086.
- **Grafana Container:** Chịu trách nhiệm hiển thị giao diện biểu đồ, lắng nghe ở cổng 3000 và liên kết trực tiếp với InfluxDB qua mạng nội bộ ảo của Docker (Docker Network)

2.7 Nền tảng lập trình Android và Dịch vụ nền (Background Service)

2.7.1 Ngôn ngữ Kotlin và Kiến trúc ứng dụng

Đồ án sử dụng ngôn ngữ lập trình **Kotlin**, hiện là ngôn ngữ được Google khuyến nghị chính thức cho phát triển Android. Kotlin cung cấp các ưu điểm vượt trội về khả năng an toàn bộ nhớ (Null Safety), giúp giảm thiểu lỗi NullPointerException thường gặp trong quá trình vận hành ứng dụng giám sát liên tục.

Về kiến trúc, ứng dụng được thiết kế theo mô hình **Single Activity**, sử dụng MainActivity làm container chính và điều hướng các màn hình chức năng thông qua Fragment (HomeFragment, ChartFragment). Kiến trúc này giúp tối ưu hóa việc quản lý vòng đời ứng dụng và tài nguyên bộ nhớ.

2.7.2 Cơ chế Foreground Service

Trong các hệ điều hành Android hiện đại (từ Android 8.0 trở lên), hệ thống áp dụng cơ chế quản lý năng lượng nghiêm ngặt, thường xuyên chấm dứt các tiến trình chạy ngầm (Background Process) để tiết kiệm pin. Điều này gây trở ngại lớn cho các ứng dụng IoT cần duy trì kết nối liên tục.

Để giải quyết vấn đề này, đồ án triển khai **Foreground Service**.

- **Nguyên lý hoạt động:** Foreground Service được hệ điều hành ưu tiên tài nguyên cao hơn các dịch vụ thông thường. Nó yêu cầu hiển thị một thông báo (Notification) cố định trên thanh trạng thái (status bar) để người dùng nhận biết ứng dụng đang hoạt động.
- **Vai trò trong hệ thống:** Service này chịu trách nhiệm khởi tạo và duy trì phiên làm việc (Session) với MQTT Broker, lắng nghe các bản tin từ topic cảm biến và kích hoạt cảnh báo rung/âm thanh ngay cả khi thiết bị di động đang ở chế độ ngủ (Sleep Mode) hoặc ứng dụng đã bị đóng giao diện.

2.7.3 Xử lý bất đồng bộ (Asynchronous Processing)

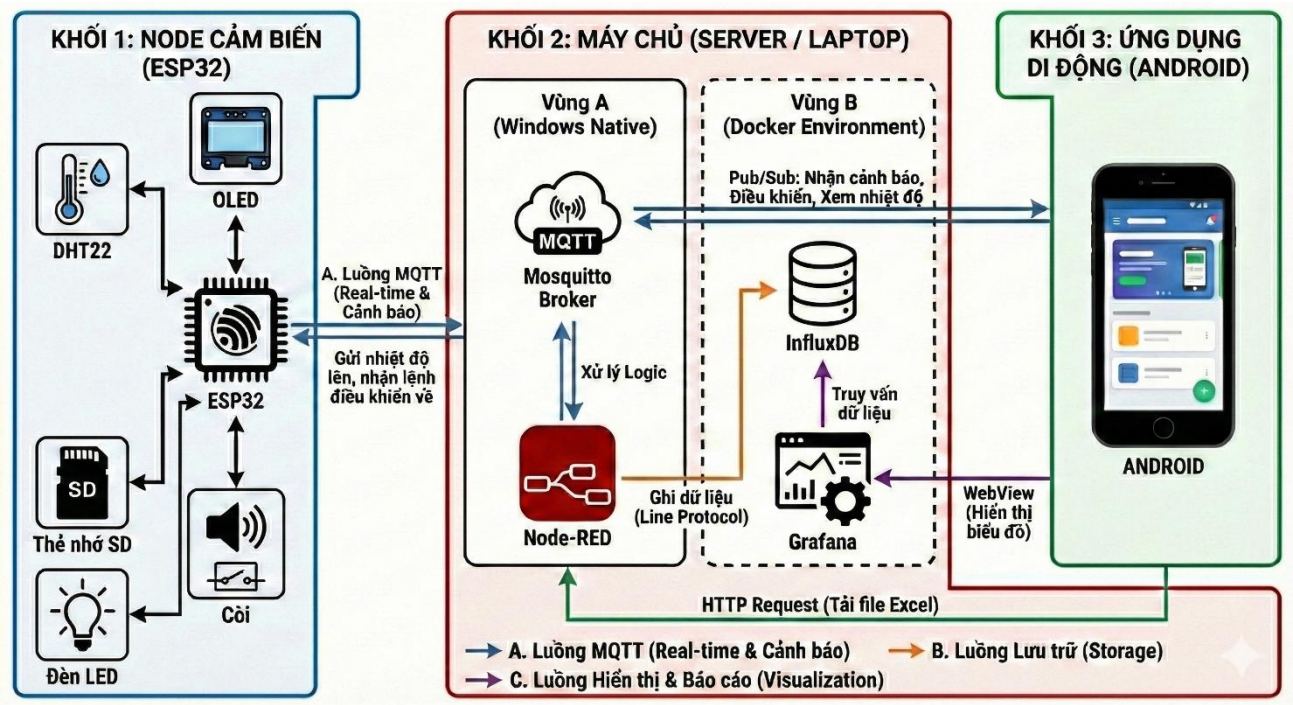
Để đảm bảo trải nghiệm người dùng mượt mà, các tác vụ mạng (Network Operations) như kết nối MQTT, gửi nhận dữ liệu được thực hiện bất đồng bộ thông qua thư viện **Paho MQTT Client**. Các sự kiện kết nối thành công, thất bại hay mất kết nối đột ngột được xử lý thông qua cơ chế Callback, đảm bảo luồng giao diện chính (Main Thread) không bị tắc nghẽn (Block), tránh hiện tượng "treo" ứng dụng (ANR - Application Not Responding).

CHƯƠNG 3 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Kiến trúc hệ thống

Hệ thống được thiết kế theo mô hình kiến trúc IoT phân tán, gồm 3 khối chính tương tác qua giao thức MQTT:

- **Khối Node Cảm biến (Sensor Node)**:Chịu trách nhiệm thu thập dữ liệu môi trường, hiển thị tại chỗ và gửi về trung tâm.
- **Khối Server (Local Gateway)**: Đóng vai trò Broker trung chuyển tin nhắn, xử lý logic lưu trữ và trực quan hóa.
- **Khối Ứng dụng di động (Mobile Client)**: Giao diện giám sát từ xa và nhận cảnh báo.



Hình ảnh 3.1 Sơ đồ kiến trúc hệ thống

3.2 Thiết kế chi tiết phần cứng

Dựa trên yêu cầu về độ ổn định và khả năng hoạt động độc lập, sơ đồ đấu nối phần cứng được thiết kế như sau:

Bảng phân bố chân kết nối (Pin Mapping)

Linh kiện	Chân linh kiện	Chân ESP32 (GPIO)	Chức năng	Ghi chú
DHT22	DATA	GPIO 15	Đọc dữ liệu	Có trở kéo lên 4.7kΩ
OLED SSD1306	SDA	GPIO 21	Giao tiếp I2C	Địa chỉ 0x3C
	SCL	GPIO 22	Giao tiếp I2C	
Thẻ nhớ MicroSD	CS (Chip Select)	GPIO 5	Giao tiếp SPI	Lưu log khi mất mạng
	MOSI	GPIO 23	Giao tiếp SPI	
	MISO	GPIO 19	Giao tiếp SPI	
	SCK	GPIO 18	Giao tiếp SPI	
Còi/Relay	SIG	GPIO 4	Điều khiển Output	Kích mức cao (High)

Nguyên lý cấp nguồn:

Hệ thống sử dụng module Battery Shield V3 tích hợp pin Li-ion 18650.

- Điện áp pin (3.7V - 4.2V) được nâng áp (Boost) lên 5V cấp cho chân VIN của ESP32.
- Tích hợp mạch sạc TP4056 qua cổng MicroUSB, cho phép vừa sạc vừa dùng.

3.3 Kiến trúc hệ thống Backend và Luồng dữ liệu (Data flow)

Hệ thống Backend được thiết kế theo mô hình vi dịch vụ (Microservices), triển khai trên hạ tầng máy chủ cục bộ (Local Host) sử dụng công nghệ ảo hóa Docker. Kiến trúc này đảm bảo tính độc lập giữa các thành phần, khả năng mở rộng linh hoạt và tối ưu hóa việc quản lý tài nguyên hệ thống.

3.3.1 Sơ đồ luồng dữ liệu (Data Flow Diagram)

Quy trình xử lý dữ liệu trong hệ thống tuân theo trình tự khép kín sau:

1. **Thu thập (Acquisition):** Node cảm biến (ESP32) thu thập dữ liệu nhiệt độ và độ ẩm từ cảm biến DHT22, đóng gói thành các bản tin (Payload) riêng biệt và gửi qua giao thức MQTT.
2. **Trung chuyển (Transportation):** MQTT Broker (Mosquitto) đóng vai trò trung gian, tiếp nhận các bản tin từ ESP32 và phân phối đến các thành phần đã đăng ký (Subscribe).
3. **Xử lý logic và Lưu trữ (Processing & Storage):**
   - Dữ liệu thời gian thực từ các topic phong_hoc/nhiet_do và phong_hoc/do_am được Node-RED tiếp nhận và đưa qua khối hàm chức năng (Insert_Data_InfluxDB) để định dạng lại cấu trúc JSON tiêu chuẩn trước khi ghi trực tiếp vào cơ sở dữ liệu InfluxDB.
   - Đồng thời, Node-RED lắng nghe topic phong_hoc/lich_su để xử lý các gói tin được ESP32 gửi bù sau sự cố mất mạng, đảm bảo tính toàn vẹn của dữ liệu lịch sử.
4. **Thuật toán Hợp nhất dữ liệu phục vụ Báo cáo (Data Aggregation for Reporting):**
   - Khi người dùng yêu cầu xuất file Excel từ ứng dụng di động (thông qua http in node), Node RED sẽ truy vấn dữ liệu thô từ InfluxDB.
   - Tại đây, các khối xử lý Group Data và Fix Time được kích hoạt để thực hiện thuật toán đồng bộ: Nhóm các giá trị nhiệt độ và độ ẩm có mốc thời gian gần nhau (trong khoảng sai số cho phép) thành một dòng dữ liệu duy nhất, loại bỏ hiện tượng lệch pha thời gian trước khi chuyển đổi sang định dạng CSV.
5. **Trực quan hóa (Visualization - Grafana on Docker):** Grafana (chạy trên Docker Container) truy vấn dữ liệu từ InfluxDB thông qua cổng nội bộ, hiển thị dưới dạng biểu đồ đường (Graph) và bảng (Table).

3.3.2 Thiết kế chi tiết các Service

Service MQTT Broker: Sử dụng Mosquitto, hoạt động tại cổng 1883. Đây là trung tâm truyền thông của toàn hệ thống, chịu trách nhiệm duy trì kết nối thời gian thực với thiết bị phần cứng và ứng dụng di động.

Service Node-RED:

- **Input Nodes:** Lắng nghe topic phong_hoc/nhiet_do và phong_hoc/do_am.
- **Function Nodes:** Thực hiện logic kiểm tra ngưỡng an toàn và kích hoạt còi báo động (nếu cần).
- **Output Nodes:** Xuất dữ liệu sang InfluxDB và cung cấp API HTTP (/xuat-bao-cao) phục vụ chức năng xuất báo cáo định dạng CSV cho ứng dụng Android.

3.4 Kiến trúc phần mềm Android

Ứng dụng di động (Mobile Client) được phát triển trên nền tảng **Android Native** sử dụng ngôn ngữ **Kotlin**. Kiến trúc phần mềm được thiết kế theo mô hình hướng dịch vụ (ServiceOriented), tách biệt rõ ràng giữa tầng giao diện người dùng (Presentation Layer) và tầng xử lý nghiệp vụ nền (Background Logic Layer).

3.4.1 Mô hình Single Activity - Multiple Fragments

Ứng dụng được thiết kế dựa trên mô hình **Single Activity - Multiple Fragments** nhằm tối ưu hóa việc quản lý bộ nhớ và vòng đời ứng dụng (Application Lifecycle).

- **Activity Chính (MainActivity):** Đóng vai trò là container duy nhất, chịu trách nhiệm khởi tạo môi trường thực thi và quản lý ngăn xếp điều hướng (Navigation Stack).
- **Các Phân hệ Giao diện (Fragments):** Các chức năng nghiệp vụ được module hóa thành các Fragment độc lập, bao gồm HomeFragment (Giám sát và Điều khiển) và ChartFragment (Phân tích dữ liệu). Việc chuyển đổi giữa các Fragment không yêu cầu khởi tạo lại kết nối mạng, giúp duy trì phiên làm việc liên tục với máy chủ.

3.4.2 Thiết kế luồng xử lý nền và Cảnh báo thời gian thực

Để khắc phục cơ chế quản lý năng lượng (Doze Mode) của hệ điều hành Android - vốn có xu hướng chấm dứt các tiến trình nền để tiết kiệm pin - hệ thống triển khai lớp MqttService dưới dạng **Foreground Service** (Dịch vụ tiền cảnh).

- **Cơ chế duy trì:** Dịch vụ này được gán độ ưu tiên cao trong hệ thống thông qua việc hiển thị một thông báo thường trực (Persistent Notification) trên thanh trạng thái. Điều này đảm bảo tiến trình kết nối MQTT không bị hệ điều hành thu hồi tài nguyên ngay cả khi ứng dụng chuyển sang trạng thái nghỉ.

- **Xử lý Cảnh báo:** Logic so sánh ngưỡng nhiệt độ và kích hoạt bộ rung (Vibrator) được thực thi trực tiếp tại Service này, tách biệt hoàn toàn với lớp giao diện (UI Layer), đảm bảo tính tin cậy của hệ thống cảnh báo.

3.4.3 Tích hợp Trực quan hóa Dữ liệu (WebView Integration)

Module ChartFragment thực hiện việc nhúng giao diện biểu đồ từ hệ thống Grafana thông qua thành phần **WebView**.

- **Cấu hình Kiosk Mode:** URL truy cập dashboard được cấu hình tham số &kiosk, loại bỏ hoàn toàn các thành phần điều hướng của trình duyệt để tích hợp liền mạch vào giao diện ứng dụng Native.
- **Tối ưu hóa Hiệu năng:** Việc sử dụng WebView cho phép chuyển giao tải trọng tính toán và vẽ đồ thị (Rendering) cho Server, giảm thiểu mức tiêu thụ tài nguyên CPU và RAM trên thiết bị di động.

3.4.4 Cơ chế Đồng bộ Trạng thái (State Synchronization)

Hệ thống giải quyết vấn đề đồng bộ dữ liệu giữa thiết bị phần cứng (ESP32) và ứng dụng di động thông qua cơ chế **Retained Messages** của giao thức MQTT.

Lưu giữ Cấu hình: Các lệnh cài đặt ngưỡng cảnh báo (phong_hoc/nguong_canh_bao) hoặc trạng thái thiết bị (phong_hoc/coi) được gửi đi với cờ retained = true. Điều này đảm bảo trạng thái hệ thống được lưu trữ bền vững tại Broker và cập nhật ngay lập tức cho bất kỳ thiết bị nào mới tham gia vào mạng lưới, loại bỏ độ trễ đồng bộ khi khởi động lại ứng dụng.

CHƯƠNG 4 XÂY DỰNG THUẬT TOÁN VÀ CÀI ĐẶT

4.1 Giải thuật điều khiển trên Node cảm biến (ESP32)

Phần mềm nhúng trên thiết bị ESP32 được xây dựng dựa trên cơ chế máy trạng thái (State Machine), ưu tiên tính ổn định và khả năng tự phục hồi (Fail-safe) khi gặp sự cố về mạng hoặc nguồn điện.

4.1.1 Lưu đồ thuật toán tổng quát

Hệ thống hoạt động theo vòng lặp vô hạn với chu kỳ lấy mẫu được định nghĩa trước (Sampling Interval). Quy trình xử lý chính bao gồm các bước:

1. **Khởi tạo (Initialization):** Thiết lập các giao tiếp ngoại vi (I2C cho OLED, SPI cho thẻ nhớ SD, GPIO cho cảm biến/còi).
2. **Kết nối mạng:** Thực hiện kết nối WiFi và MQTT Broker. Nếu thất bại, hệ thống tự động chuyển sang chế độ hoạt động ngoại tuyến (Offline Mode).
3. **Thu thập dữ liệu:** Đọc giá trị từ cảm biến DHT22, kiểm tra tính hợp lệ của dữ liệu (loại bỏ giá trị NaN).
4. **Ra quyết định (Decision Making):** So sánh giá trị đo được với ngưỡng cảnh báo (TEMP_LIMIT, HUM_LIMIT) để kích hoạt còi báo động tại chỗ.

4.1.2 Cơ chế ghi dữ liệu ngoại tuyến (Data Persistence)

Để đảm bảo tính toàn vẹn dữ liệu (Data Integrity) trong môi trường mạng không ổn định, hệ thống triển khai giải thuật tự động chuyển mạch lưu trữ:

- **Trạng thái Online:** Dữ liệu được đóng gói thành chuỗi (String) và gửi trực tiếp lên MQTT Broker thông qua các topic phong_hoc/nhiet_do và phong_hoc/do_am.
- **Trạng thái Offline:** Khi phát hiện mất kết nối (!client.connected()), hàm logToSD() được kích hoạt.
    - Hệ thống truy xuất thời gian thực (Epoch Time) từ bộ đếm nội bộ.
    - Dữ liệu được ghi nối tiếp (Append) vào tệp offline_log.txt trên thẻ nhớ SD dưới định dạng CSV: Timestamp,NhietDo,DoAm.
    - **Cơ chế tự phục hồi SPI:** Trong trường hợp giao tiếp với thẻ nhớ bị lỗi (do nhiễu hoặc lỏng kết nối), hệ thống thực hiện quy trình khởi động mềm (SD.end() và spiSD.begin()) để khôi phục khả năng ghi dữ liệu mà không cần khởi động lại toàn bộ thiết bị.

4.1.3 Thuật toán đồng bộ lại dữ liệu (Data Synchronization)

Ngay khi kết nối mạng được khôi phục, hàm syncOfflineData() sẽ được thực thi để đồng bộ dữ liệu quá khứ lên Server:

1. Kiểm tra sự tồn tại của tệp nhật ký ngoại tuyến.
2. Đọc tuần tự từng dòng dữ liệu và gửi lên topic phong_hoc/lich_su.
3. Sau khi đồng bộ hoàn tất, tệp nhật ký sẽ được xóa để giải phóng bộ nhớ, chuẩn bị cho chu kỳ hoạt động tiếp theo.

4.2 Xử lý luồng dữ liệu tại Backend (Node-RED)

Hệ thống Backend sử dụng Node-RED làm trung tâm xử lý logic (Logic Processing Unit), thực hiện hai nhiệm vụ chính: Định tuyến dữ liệu thời gian thực và Hợp nhất dữ liệu phục vụ báo cáo

Ứng dụng Android giải quyết hai bài toán khó: Giữ kết nối ngầm và Đồng bộ giao diện.

4.2.1 Quy trình xử lý dữ liệu thời gian thực

Dựa trên lưu đồ xử lý (Node-RED Flow), dữ liệu đi theo luồng sau:

- **Input:** Các node MQTT in lắng nghe topic nhiệt độ và độ ẩm.
- **Processing:** Dữ liệu đi qua các node chức năng để kiểm tra định dạng.
- **Storage:** Hàm Insert_Data_InfluxDB chuyển đổi dữ liệu sang cấu trúc Line Protocol của InfluxDB và thực hiện ghi dữ liệu thông qua node influxdb out. Quá trình này diễn ra với độ trễ thấp (<100ms) để phục vụ giám sát tức thời.

4.2.2 Thuật toán hợp nhất dữ liệu (Data Aggregation Algorithm)

Một thách thức kỹ thuật khi sử dụng giao thức MQTT là các gói tin nhiệt độ và độ ẩm được gửi tách biệt tại các thời điểm khác nhau, dẫn đến việc dữ liệu trong cơ sở dữ liệu bị phân tán (rời rạc). Để xuất báo cáo Excel (CSV) chuẩn hóa, nhóm thực hiện xây dựng thuật toán hợp nhất dữ liệu trong Node-RED:

1. **Truy vấn (Query):** Node http in nhận yêu cầu từ ứng dụng Android, kích hoạt truy vấn dữ liệu thô từ InfluxDB trong khoảng thời gian chỉ định.
2. **Nhóm dữ liệu (Grouping):** Sử dụng node chức năng Group Data để quét qua toàn bộ tập dữ liệu. Các điểm dữ liệu nhiệt độ và độ ẩm có sai lệch thời gian nhỏ (trong khoảng $t < 1s$) được coi là cùng một thời điểm đo.
3. **Hợp nhất (Merging):** Node Fix Time thực hiện gộp hai bản ghi rời rạc thành một đối tượng dữ liệu duy nhất: {time: T, temp: X, humid: Y}.
4. **Định dạng (Formatting):** Dữ liệu sau khi hợp nhất được chuyển đổi sang định dạng CSV và trả về cho ứng dụng Client tải xuống.

4.3 Giải thuật và quy trình xử lý ứng dụng Android

Ứng dụng di động không chỉ đóng vai trò hiển thị mà còn thực hiện các logic xử lý thời gian thực và đồng bộ hóa trạng thái hệ thống. Các giải thuật được xây dựng dựa trên mô hình lập trình hướng sự kiện (Event-driven Programming).

4.3.1 Giải thuật Giám sát và Cảnh báo nền

Đây là quy trình xử lý quan trọng nhất, đảm bảo tính đáp ứng thời gian thực ngay cả khi thiết bị ở chế độ nghỉ (Sleep Mode). Logic được thực thi trong MqttService theo trình tự sau:

1. **Khởi tạo (Initialization):**
    - Hệ thống khởi tạo một đối tượng MqttAndroidClient độc lập với luồng giao diện (UI Thread).
    - Thiết lập kênh thông báo (Notification Channel) với độ ưu tiên cao để đảm bảo quyền truy cập tài nguyên hệ thống.
    - Đăng ký lắng nghe (Subscribe) hai topic chủ chốt: phong_hoc/nhiet_do (dữ liệu cảm biến) và phong_hoc/tat_tieng (lệnh điều khiển).
2. **Vòng lặp xử lý sự kiện (Event Processing Loop):**
   Khi một gói tin MQTT (messageArrived) được tiếp nhận, hệ thống thực hiện phân luồng xử lý:
    - **Trường hợp 1: Nhận dữ liệu nhiệt độ (T_VAL), độ ẩm (H_VAL):**
        - Chuyển đổi dữ liệu (Parsing) từ định dạng String sang Float.
        - So sánh với ngưỡng cài đặt cục bộ T_LIMIT, H_LIMIT
        - **Điều kiện kích hoạt:** Nếu T_VAL > T_LIMIT hoặc H_VAL > H_LIMIT
            - Cập nhật đối tượng Notification với giao diện cảnh báo
            - Kích hoạt Bộ rung (Vibrator) theo chu kỳ định sẵn (Pattern: 500ms rung - 500ms nghỉ).
        - **Điều kiện bình thường:** Nếu T_VAL ≤ T_LIMIT và H_VAL ≤ H_LIMIT
            - Ngắt Bộ rung (nếu đang hoạt động).
            - Cập nhật Notification về trạng thái giám sát tiêu chuẩn.
    - **Trường hợp 2: Nhận tín hiệu điều khiển:**
        - Nếu Mute_State == "MUTE": Thực hiện ngắt mềm (Software Interrupt), cưỡng chế dừng Bộ rung ngay lập tức thông qua hàm vibrator.cancel().
        - Nếu Mute_State == "UNMUTE": Khôi phục trạng thái cho phép rung trong chu kỳ cảnh báo tiếp theo.

4.3.2 Logic Kiểm tra An toàn Tổng hợp

Tại tầng giao diện (HomeFragment), ứng dụng triển khai thuật toán đánh giá rủi ro đa biến để cung cấp phản hồi trực quan cho người dùng.

- **Đầu vào:**
  - Nhiệt độ hiện tại T_VAL và Ngưỡng nhiệt độ T_LIMIT.
  - Độ ẩm hiện tại H_VAL và Ngưỡng độ ẩm H_LIMIT.
- **Thuật toán đánh giá:**

  Trạng thái hệ thống (Status) được xác định bởi biểu thức logic:

  DANGER = (T_VAL > T_LIMIT) or (H_VAL > H_LIMIT)

- **Hành vi hệ thống:**
  - Nếu DANGER là True: Hiển thị cảnh báo cục bộ trên Dashboard, kích hoạt hiển thị nút chức năng "Tắt tiếng còi" (btnMute) và thay đổi màu sắc hiển thị các thông số sang màu đỏ để thu hút sự chú ý.
  - Nếu DANGER là False: Duy trì giao diện ở trạng thái an toàn (Safe State), ẩn các nút chức năng khẩn cấp để tối giản hóa giao diện.

4.3.3 Cơ chế Đồng bộ hóa Cấu hình

Để đảm bảo tính nhất quán dữ liệu (Data Consistency) trong môi trường phân tán (nhiều thiết bị cùng điều khiển), ứng dụng sử dụng cơ chế đồng bộ dựa trên bản tin lưu giữ (Retained Messages):

1. **Quy trình Ghi (Write Operation):** Khi người dùng thay đổi tham số cấu hình (ví dụ: Ngưỡng cảnh báo), ứng dụng đóng gói giá trị mới và gửi đi với cờ retained = true. Điều này chỉ thị cho MQTT Broker lưu trữ giá trị này vĩnh viễn trong bộ nhớ đệm.
2. **Quy trình Đọc và Cập nhật (Read & Update Operation):** Ngay khi khởi động, ứng dụng tự động đăng ký các topic trạng thái (phong_hoc/nguong_hien_tai). Broker sẽ lập tức trả về giá trị retained gần nhất. Ứng dụng cập nhật giá trị này vào biến cục bộ và đồng bộ lên các thành phần giao diện (Switch, Seekbar), đảm bảo người dùng luôn nhìn thấy trạng thái thực tế của hệ thống.

4.3.4 Giải thuật Tích hợp Hiển thị Lai

Module biểu đồ (ChartFragment) sử dụng kỹ thuật "Offloading" (Giảm tải) để tối ưu hóa hiệu năng thiết bị di động:

- Thay vì thực hiện việc truy vấn cơ sở dữ liệu và vẽ đồ thị (Rendering) ngay trên điệnthoại (tốn tài nguyên CPU/RAM), ứng dụng sử dụng WebView để tải giao diện đãđược render sẵn từ Server Grafana.
- **Kỹ thuật xử lý URL:** Ứng dụng tự động chèn tham số &kiosk vào đường dẫn truy cập. Tham số này kích hoạt chế độ hiển thị nhúng trên Grafana, loại bỏ toàn bộ các thanh công cụ (Toolbar) và điều hướng (Navigation Bar) của trình duyệt, giúp biểu đồ hiển thị liền mạch với giao diện Native của ứng dụng Android.

CHƯƠNG 5 KẾT QUẢ THỰC HIỆN VÀ ĐÁNH GIÁ

5.1 Kết quả phần cứng

Sau quá trình thiết kế và thi công, nhóm đã hoàn thiện trạm quan trắc (Sensor Node) với các đặc điểm sau:

- **Đóng gói:** Thiết bị được tích hợp gọn gàng trong hộp kỹ thuật kích thước 120x70x40mm.
- **Giao diện vật lý:**
  - Màn hình OLED 0.96 inch hiển thị thông số thời gian thực và trạng thái kết nối (WiFi/MQTT).
  - Đèn LED chỉ thị trạng thái và Còi báo động được bố trí ở mặt trên để dễ quan sát.

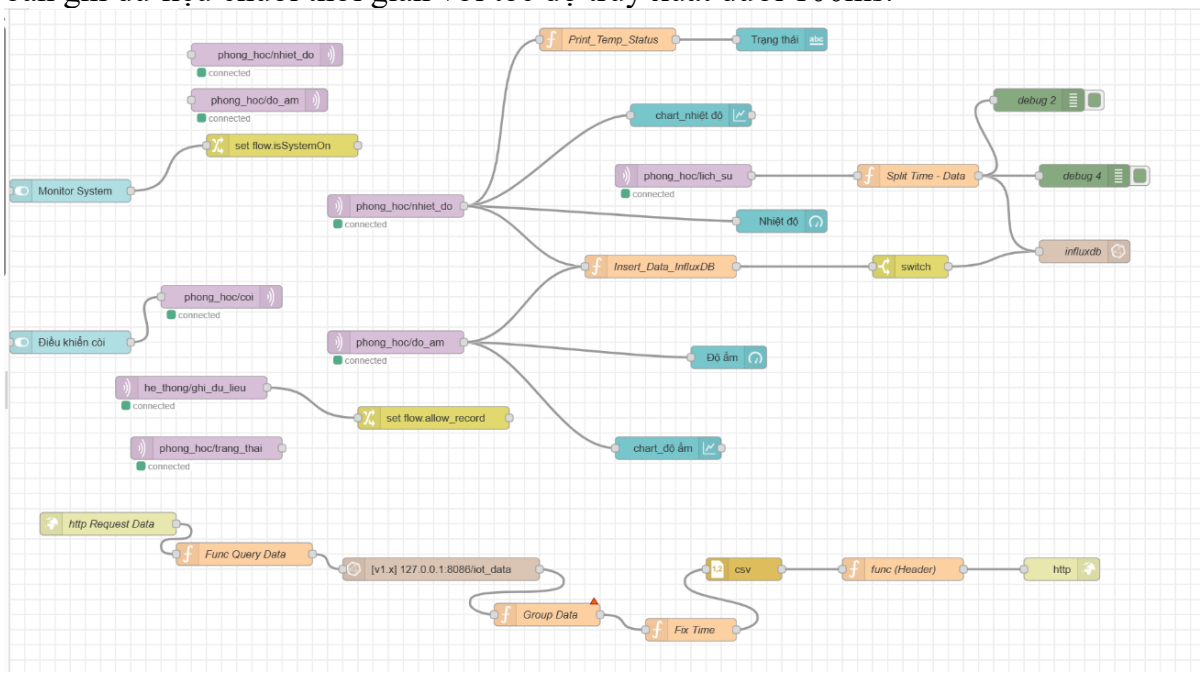
Nguồn điện: Sử dụng pin Li-ion 18650 tích hợp mạch sạc, cho phép thiết bị hoạt động không dây hoàn toàn.

Hình ảnh 5.1 Hộp cảm biến và cảnh báo

5.2 Kết quả phần mềm

5.2.1 Hệ thống Backend và Xử lý dữ liệu

Hệ thống máy chủ cục bộ (Local Server) đã được triển khai thành công trên nền tảng Docker, đảm bảo sự cô lập môi trường và tính ổn định cao.

- **Node-RED:** Đã xây dựng hoàn chỉnh lưu đồ xử lý (Flow) phức tạp, bao gồm các chức năng: Nhận dữ liệu MQTT, Ghi dữ liệu vào InfluxDB, Truy vấn và Gộp dữ liệu (Data Aggregation) để xuất báo cáo.
- **Cơ sở dữ liệu:** InfluxDB và Grafana hoạt động đồng bộ, cho phép lưu trữ hàng triệu bản ghi dữ liệu chuỗi thời gian với tốc độ truy xuất dưới 100ms.
- 

Hình ảnh 5.2.1 Lưu đồ luồng xử lý dữ liệu trên Node-RED

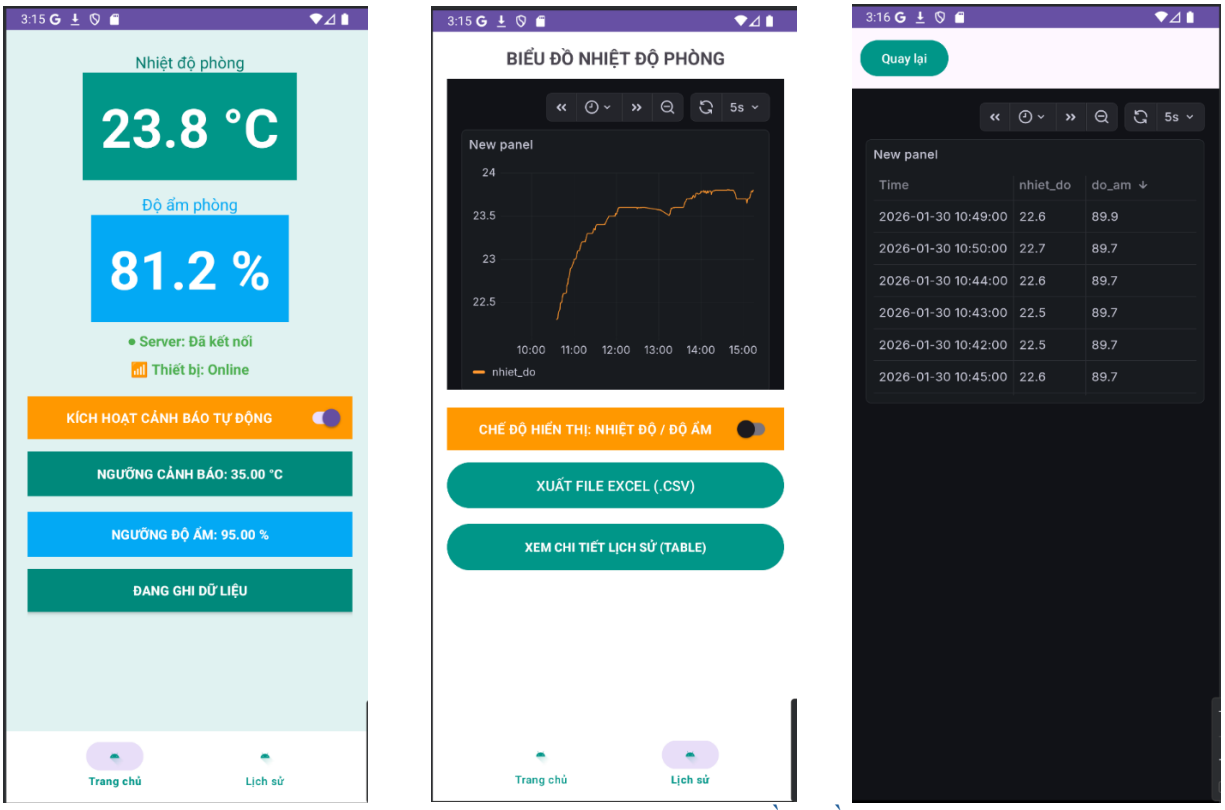
5.2.2 Ứng dụng Android

Ứng dụng trên điện thoại hoạt động mượt mà với các tính năng chính đã hoàn thiện:

- **Dashboard giám sát:**
  - Hiển thị thẻ màu trực quan: **Màu xanh** (An toàn) và **Màu đỏ** (Cảnh báo) dựa trên ngưỡng cài đặt.
  - Trạng thái kết nối Server và Thiết bị được cập nhật theo thời gian thực (Online/Offline).
- **Cài đặt ngưỡng (Threshold Setting):**
  - Người dùng có thể thay đổi ngưỡng nhiệt độ/độ ẩm trực tiếp trên App. Giá trị này được gửi qua MQTT (*phong_hoc/nguong_canh_bao*) để đồng bộ xuống ESP32.
- **Dịch vụ chạy nền (Background Service):**

- Thanh thông báo (Notification) "Đang giám sát hệ thống IoT" luôn hiển thị, đảm bảo App không bị hệ điều hành tắt ngầm.
- Khi có cảnh báo, điện thoại rung theo nhịp (500ms rung - 500ms nghỉ) ngay cả khi tắt màn hình.

- **Biểu đồ lịch sử:**
  - Tích hợp thành công biểu đồ Grafana vào WebView, cho phép xem lại diễn biến nhiệt độ trong quá khứ.



Hình ảnh 5.2.2.a/b/c Giao diện phần mềm Android

5.3 Đánh giá hiệu năng hệ thống

Bảng 5.1. Bảng đánh giá thông số kỹ thuật thực tế

Tiêu chí	Kết quả đo đạc	Đánh giá
Độ trễ	Trung bình **300ms - 500ms** (Trong mạng LAN).	Rất tốt (Đáp ứng thời gian thực).
Tính toàn vẹn dữ liệu	**100%** dữ liệu được khôi phục từ thẻ nhớ SD sau khi mạng được kết nối lại.	Cơ chế "Fail-safe" hoạt động đúng thiết kế.
Độ tin cậy kết nối	Cơ chế tự động kết nối lại (Auto Reconnect) hoạt động tốt khi WiFi chập chờn.	Đạt yêu cầu cho một buổi học.
Khoảng cách kết nối	Ổn định trong bán kính **15m** (xuyên 1 lớp tường).	Phù hợp không gian phòng học.
Khả năng đồng bộ	Trạng thái nút bấm trên App và thiết bị đồng bộ trong **< 1 giây**.	Ổn định.
Dữ liệu báo cáo	File CSV xuất ra thẳng hàng, không bị lỗi so le thời gian.	Đạt yêu cầu quản lý.

Kết quả thực nghiệm cho thấy hệ thống đã đáp ứng hoàn toàn các yêu cầu kỹ thuật đề ra trong đồ án, tạo nên một giải pháp giám sát phòng học toàn diện, ổn định và dễ sử dụng

CHƯƠNG 6 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Đồ án **"Hệ thống giám sát an toàn phòng học IoT"** đã hoàn thành 100% khối lượng công việc đề ra ban đầu. Nhóm thực hiện đã đạt được các kết quả quan trọng:

1. **Làm chủ kiến trúc Full-stack IoT:** Đã xây dựng thành công một hệ thống khép kín từ tầng vật lý (ESP32/Sensors) đến tầng xử lý trung tâm (Docker/Node-RED) và tầng ứng dụng người dùng (Android Native) **Giải quyết vấn đề thực tế:** Hệ thống hoạt động ổn định, cung cấp giải pháp giám sát nhiệt độ/độ ẩm độc lập, chi phí thấp, thay thế hiệu quả cho các nhiệt kế truyền thống.
2. **Giải quyết triệt để bài toán ổn định hệ thống:** Đã thiết kế và cài đặt thành công các cơ chế an toàn (Fail-safe) như: tự động ghi thẻ nhớ khi mất mạng, đồng bộ dữ liệu ngoại tuyến (Offline Sync) và dịch vụ chạy nền (Foreground Service) trên Android.
3. **Ứng dụng công nghệ ảo hóa (Containerization):** Việc triển khai InfluxDB và Grafana trên nền tảng Docker đã chứng minh hiệu quả trong việc quản lý tài nguyên, đơn giản hóa quy trình triển khai và đảm bảo tính nhất quán của môi trường vận hành.
4. **Tối ưu hóa trải nghiệm người dùng:** Ứng dụng di động không chỉ hiển thị thông số khô khan mà còn tích hợp các biểu đồ phân tích trực quan (thông qua WebView) và cơ chế cảnh báo rung thời gian thực, mang lại trải nghiệm sử dụng chuyên nghiệp.

6.2 Hạn chế

Bên cạnh các kết quả đạt được, hệ thống vẫn tồn tại một số hạn chế cần được khắc phục trong các phiên bản tiếp theo:

- **Phạm vi kết nối:** Do hệ thống Backend (MQTT Broker, Node-RED) hiện đang được triển khai trên máy chủ cục bộ (Localhost/Laptop), khả năng giám sát từ xa qua Internet (4G/5G) bị giới hạn trong mạng nội bộ (LAN).
- **Quản lý năng lượng:** Thiết bị phần cứng chưa được tối ưu hóa sâu về năng lượng (chưa áp dụng chế độ Deep Sleep giữa các chu kỳ đo), dẫn đến thời lượng pin chỉ đạt khoảng 6-8 giờ hoạt động liên tục, chưa phù hợp cho việc triển khai dài ngày mà không có nguồn điện lưới.
- **Bảo mật:** Hiện tại giao thức MQTT chưa được kích hoạt tính năng xác thực (Username/Password) và mã hóa (TLS/SSL), tiềm ẩn rủi ro nếu triển khai trong môi trường mạng công cộng.

6.3 Hướng phát triển

Để nâng cấp đồ án thành một sản phẩm thương mại hoặc triển khai thực tế quy mô lớn, nhóm đề xuất các hướng nghiên cứu và phát triển sau:

- **Chuyển đổi hạ tầng lên Cloud (Cloud Migration):**
  - Đưa toàn bộ hệ thống Backend (Docker Containers) lên triển khai trên máy chủ ảo (VPS) hoặc các nền tảng đám mây như AWS/Google Cloud. Điều này sẽ giải quyết triệt để vấn đề kết nối từ xa.
- **Tối ưu hóa năng lượng (Low Power Optimization):**
  - Nghiên cứu lại thiết kế phần cứng và Firmware để ứng dụng chế độ ngủ sâu (Deep Sleep) cho ESP32. Mục tiêu là thiết bị chỉ "thức dậy" đo đạc trong 2 giây rồi ngủ 5 phút, kéo dài tuổi thọ pin lên hàng tuần hoặc hàng tháng.
- **Nâng cấp Bảo mật và Quản lý thiết bị:**
  - Tích hợp cơ chế xác thực người dùng (Authentication) trên App Android.
  - Mã hóa toàn bộ gói tin truyền nhận bằng giao thức MQTTS (Secure MQTT).
- **Phân tích dữ liệu nâng cao (Advanced Analytics):**
  - Sử dụng dữ liệu lịch sử lưu trong InfluxDB để huấn luyện các mô hình học máy (Machine Learning) đơn giản, nhằm dự báo xu hướng nhiệt độ hoặc phát hiện các bất thường của môi trường phòng học sớm hơn.
- **Nâng cấp App:** Bổ sung tính năng đăng nhập, phân quyền người dùng và quản lý nhiều phòng học cùng lúc.

TÀI LIỆU THAM KHẢO

[1] Espressif Systems, *ESP32 Series Datasheet*, 2023.

[2] HiveMQ, *MQTT Essentials - A Lightweight IoT Protocol*.

[3] Espressif Systems, "ESP32 Series Datasheet (Version 4.0)," 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[4] Aosong Electronics, "DHT22 (AM2302) Temperature and Humidity Module Datasheet," 2019.

[5] OASIS Standard, "MQTT Version 3.1.1 Plus Errata 01," 2015. [Online]. Available: http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html

[6] Android Developers, "Foreground services," Google Developers Documentation. [Online]. Available: https://developer.android.com/guide/components/foreground-services

[7] Android Developers, "Guide to App Architecture," Google Developers Documentation. [Online]. Available: https://developer.android.com/topic/architecture

[8] InfluxData, "InfluxDB Documentation," 2023. [Online]. Available: https://docs.influxdata.com/influxdb/

[9] Grafana Labs, "Grafana Documentation," 2023. [Online]. Available: https://grafana.com/docs/grafana/latest/

[10] Node-RED, "Node-RED Documentation & User Guide," OpenJS Foundation. [Online]. Available: https://nodered.org/docs/

[11] Docker Inc., "Docker Documentation," 2023. [Online]. Available: https://docs.docker.com/

[12] Eclipse Foundation, "Eclipse Paho MQTT Android Client," GitHub Repository. [Online]. Available: https://github.com/eclipse/paho.mqtt.android